MISUNDERSTOOD

An AI-powered platform that detects the gap between what people mean and what others understand.

Devlympics Theme: 3) Bridge the Gap
Team Size: 3 members
Project Category: Artificial Intelligence · Communication · Productivity
Project Type: Web application
Core Idea: Don't just help people communicate. Help them discover when they think they've communicated successfully but have actually misunderstood each other.

1. The problem we're solving

Have you ever experienced a conversation like this?

A team leader says:

"Make the website responsive before tomorrow."

The developer replies:

"Got it."

But the team leader expects the website to work perfectly on mobile, tablet, and desktop, while the developer only adjusts the existing desktop layout to fit a smaller screen.

Both people believe they understand the task.

Both proceed confidently.

Later, the team discovers that they were working toward different expectations.

This happens in software development, college projects, workplaces, classrooms, and everyday conversations.

The underlying problem is that **communication does not guarantee shared understanding**.

People frequently make assumptions about:

- What another person means.
- What a task actually requires.
- Which details are important.
- What someone has agreed to do.
- What words such as "soon," "finished," or "responsive" mean in a particular context.

Traditional communication tools transmit messages. AI writing assistants improve wording. Meeting assistants summarize conversations.

But a summary can be perfectly accurate while the people involved still disagree about what they should do next.

MISUNDERSTOOD aims to close that gap by identifying conflicting interpretations before they become real problems.

This concept directly fits Devlympics' *Bridge the Gap* theme, which calls for connecting people separated by differences in knowledge, understanding, language, access, or technology. Event Design Devlympics

2. Our solution

MISUNDERSTOOD is a web application that helps two or more people reach a shared understanding of a task, instruction, or decision.

It allows participants to explain what they understood independently, compares their interpretations, identifies important differences, and guides them toward an explicit agreement.

Rather than merely asking an AI chatbot to explain a sentence, users can see:

- What each person believes the instruction means.
- Where their interpretations differ.
- Which assumptions have not been confirmed.
- What questions need answers.
- What the group has finally agreed to do.

The application transforms an ambiguous conversation into a clear, mutually understood plan.

Our central promise

"Before you say 'Got it,' make sure you actually got it."

Other possible taglines:

- *Same conversation. Different realities.*
- *Catch the misunderstanding before it catches you.*
- *Less assuming. More understanding.*
- *Because "I understood" doesn't always mean "we agree."*

My recommendation is to use **"Same conversation. Different realities."** as the main branding tagline and "Before you say 'Got it,' make sure you actually got it" in the presentation.

3. How MISUNDERSTOOD works

The application has one primary workflow with six stages.

Stage 1: Create a task or conversation

One person creates a session and enters an instruction, decision, or task that needs to be understood.

For example:

"Build a login system that remembers users and lets them get back into their accounts easily."

The creator can optionally add context, such as the project's purpose, technical constraints, or deadline.

The app generates a session code that other participants can use to join.

Example session: `GAP-4821`

The application should work for both technical and nontechnical conversations, but for the first version, we should focus on tasks and instructions because they are easier to test objectively.

Stage 2: Independent interpretation

Each participant explains what they understood in their own words.

For the login-system example:

Person A — Developer

"I'll implement email/password login, session persistence, and a password-reset flow."

Person B — Project manager

"I expect users to remain logged in after closing and reopening their browser, and to be able to recover their account without contacting support."

Person C — Designer

"I'll create the login screen, a remember-me checkbox, and a forgot-password screen."

At first glance, these responses seem compatible.

But are they?

Not necessarily.

- The developer may use sessions that expire after a certain period.
- The project manager expects users to remain logged in after reopening the browser.
- The designer assumes a remember-me checkbox is required, but nobody has confirmed how it should work.

These are precisely the kinds of differences the application should investigate.

Important design decision: Participants should submit their initial interpretations independently before seeing other people's responses. Otherwise, they may simply copy the first interpretation they see, concealing the original misunderstanding.

Stage 3: AI-powered gap detection

Once everyone submits their interpretation, MISUNDERSTOOD compares the responses.

It identifies three primary types of gaps.

A. Contradiction gaps

Two participants expect incompatible outcomes.

Example:

- Person A: "The application should log users out when they close the browser."
- Person B: "Users should remain logged in after closing and reopening the browser."

These expectations need to be reconciled.

B. Assumption gaps

One participant assumes something that the others haven't explicitly agreed to.

Example:

- Person A assumes login requires an email address.
- Person B assumes users can log in using either an email address or a phone number.

Neither interpretation is necessarily unreasonable, but the requirement is unresolved.

C. Missing-information gaps

The group has not specified enough detail to make a reliable decision.

Example:

- Everyone agrees that users need password recovery.
- Nobody has decided whether recovery uses an email link, an OTP, or an administrator-assisted process.

The app should identify the missing decision rather than inventing an answer.

Stage 4: The Misunderstanding Map

This is one of the most important screens in the product.

Instead of showing a generic AI-generated paragraph, MISUNDERSTOOD presents the differences visually.

Imagine this interface:

Task: Build a persistent login system.

Topic	Developer	Project manager	Status
Login method	Email and password	Email or phone	Conflict
Session duration	Expires after a defined period	Remains active between visits	Clarification needed
Password recovery	Reset link	OTP	Conflict
Login screen	Basic login form	Login form with recovery options	Partial agreement

Each issue receives a status:

- **Conflict:** Two interpretations contradict each other.
- **Unconfirmed:** An assumption has not been verified.
- **Missing:** A decision needs to be made.

- **Aligned:** Participants agree on the requirement.

Selecting an issue opens the underlying interpretations and suggests a clarification question.

For example:

Should users be able to log in with an email address only, or with either an email address or a phone number?

Participants can then resolve the issue explicitly.

Why this matters: The output is not merely an AI summary. It is a structured view of where shared understanding breaks down.

4. Our signature feature: The Assumption Gap Detector

This is the feature I would prioritize above everything else.

Many AI tools can rewrite messages, summarize conversations, and answer questions. MISUNDERSTOOD should distinguish itself by identifying **what each person believes that the others have not actually agreed to**.

Example

The original instruction is:

"Make sure the project is ready for the presentation."

Three teammates interpret it differently.

Teammate	Interpretation
Developer	The main functionality must work.
Designer	The interface must look polished.
Presenter	The demo must be rehearsed and the slides must be ready.

These interpretations aren't necessarily contradictory. In fact, they represent different parts of the same objective.

The actual problem is that nobody has confirmed whether all three requirements are expected.

MISUNDERSTOOD identifies this as a **scope gap** and asks:

"Does 'ready for the presentation' mean a working product, a polished interface, a rehearsed demo, or all three?"

The team can select the agreed requirements.

The application then generates a shared checklist:

- Core functionality works.

- Interface is presentation-ready.
- Demo has been rehearsed.
- Presentation materials are ready.

Now everyone has a clearer understanding of the expected outcome.

How we make this technically meaningful

The AI should not simply count differences between sentences.

Instead, it should extract structured information from each participant's interpretation.

For example:

```
{
  "requirement": "Presentation readiness",
  "interpretation": {
    "functional_product": true,
    "polished_interface": false,
    "rehearsed_demo": false,
    "presentation_materials": false
  }
}
```

Another participant might have:

```
{
  "requirement": "Presentation readiness",
  "interpretation": {
    "functional_product": true,
    "polished_interface": true,
    "rehearsed_demo": true,
    "presentation_materials": true
  }
}
```

The system can compare the extracted requirements and identify differences.

These examples illustrate the internal representation, not a claim that an LLM will always extract the correct information.

The application must let participants correct inaccurate interpretations.

The important distinction: A difference is not automatically a misunderstanding. The app should distinguish between complementary perspectives, genuine contradictions, and unresolved assumptions.

That distinction will make the product more useful and credible.

5. The second signature feature: Misunderstanding Risk Score

We can add a score that helps teams understand which unresolved issues deserve attention first.

Suppose a session has five unresolved issues:

Issue	Potential impact	Status
Authentication method	High	Unresolved
Mobile layout	Medium	Unresolved
Button colour	Low	Unresolved
Password recovery	High	Unresolved
Loading animation	Low	Resolved

The app prioritizes authentication and password recovery because those decisions could significantly affect implementation.

We can label issues:

- **Critical:** Could change the core functionality or make the result unusable.
- **Important:** Could cause substantial rework or confusion.
- **Minor:** Worth clarifying but unlikely to affect the overall outcome.

How the score works

For the MVP, use a transparent, rule-based heuristic rather than pretending to predict the probability of failure.

For example:

- Impact: 1–5.
- Uncertainty: 1–5.
- Score = Impact × Uncertainty.

A high-impact issue with high uncertainty receives a score of 25. A low-impact issue with low uncertainty receives a score of 1.

These values are prioritization aids, not validated measurements of misunderstanding or project failure.

We can show the result as a prioritized issue list rather than introducing a complicated scoring dashboard.

Why this feature helps: It answers a practical question: *Which misunderstanding should we resolve first?*

6. The third signature feature: The Bridge Test

This is the feature that will make our final presentation stronger.

We should not merely tell judges that MISUNDERSTOOD improves communication. We should demonstrate a measurable difference between the original state and the resolved state.

Round 1: Before using MISUNDERSTOOD

Give two teammates the same ambiguous instruction:

"Prepare the application for launch."

Ask each person to independently list the requirements they believe this includes.

Their answers might differ:

- Developer: Deploy the application.
- Designer: Finalize the UI.
- Project manager: Test the application, deploy it, and prepare documentation.

Record the unresolved differences.

Round 2: Using MISUNDERSTOOD

Run the same workflow with the app.

1. Both teammates submit their interpretations.
2. The AI identifies missing requirements.
3. The team answers clarification questions.
4. The application generates an agreed checklist.
5. Each teammate confirms the final understanding.

The comparison screen

At the end, display:

Metric	Before clarification	After clarification
Unresolved requirements	4	0
Confirmed requirements	2	6
Shared checklist	Incomplete	Complete

The numbers are illustrative. In the actual application, calculate them from the session data.

This gives the judges a visible result and a compelling narrative.

However, don't claim that the Bridge Test proves the app prevents real-world project failures. It demonstrates that the app can help participants identify and resolve discrepancies in a controlled task.

7. Complete feature list

For a three-person team working within 24 hours, I would divide the features into three levels.

Must-have features

These define the product.

Feature	Description
Session creation	Create a task or discussion session.
Join with a code	Let other participants join the session.
Independent responses	Collect each participant's interpretation before revealing others' answers.
AI gap detection	Identify contradictions, assumptions, and missing information.
Misunderstanding Map	Show differences between participants in a structured interface.
Clarification questions	Help participants resolve specific issues.
Shared agreement	Generate a common checklist from confirmed decisions.
Confirmation	Let participants confirm the final interpretation.
Bridge Test	Compare unresolved issues before and after clarification.

Differentiating features

These make the product more distinctive.

Feature	Why it matters
Assumption Gap Detector	Finds expectations that haven't been mutually confirmed.
Issue prioritization	Highlights misunderstandings that could cause the most damage.
Evidence-based issue display	Shows the statements that caused a conflict instead of presenting an unsupported AI judgment.
Complementary-perspective detection	Avoids treating every difference in wording as a contradiction.
Shared agreement history	Records how the team resolved each issue.

Future features

These are ideas for later versions, not requirements for the hackathon.

- Live integration with meetings.
- Automatic analysis of chat conversations.
- Integration with GitHub issues and project-management tools.
- Voice-based interpretation.
- Multilingual communication support.
- Slack and Discord integration.
- Long-term analysis of recurring team communication problems.
- Organization-wide reporting on unresolved requirements.

Scope recommendation: Do not attempt to implement every feature during Devlympics. A reliable independent-response workflow, a useful Misunderstanding Map, and a working Bridge Test are enough to demonstrate the core concept.

8. Example user journey: From confusion to clarity

Let's walk through a complete session to see how the product would work.

Scenario: A college team is building an event registration website

The team leader creates a session.

Task:

"Build an event registration system that can handle registrations and prevent people from registering more than once."

Three teammates join.

Teammate 1 — Developer

"I'll create a registration form, save registrations in the database, and reject duplicate email addresses."

Teammate 2 — Designer

"I'll build the registration page and confirmation screen. Users should be able to register again if they want to correct a mistake."

Teammate 3 — Project lead

"Each student should register only once, receive a confirmation email, and be able to edit their registration details later."

What the AI discovers

Gap 1: Duplicate registration

The developer intends to reject duplicate email addresses.

The designer expects users to be able to register again to correct mistakes.

The project lead expects each student to have only one registration.

The system flags a potential conflict and asks:

"Should an existing registration be updated when a student returns, or should a second registration be rejected?"

Gap 2: Editing registrations

The project lead expects users to edit their details.

Neither the developer nor the designer has explicitly confirmed how editing should work.

The system flags an unconfirmed requirement.

Gap 3: Confirmation email

The project lead expects an email confirmation, but the other participants haven't included that feature in their plans.

The system flags a missing agreement.

The team resolves the issues

The participants agree that:

1. Each student can have one registration per event.
2. Returning students can edit their existing registration.
3. The system rejects duplicate registrations but doesn't prevent legitimate updates.
4. A confirmation email is required.

MISUNDERSTOOD produces a final specification:

Agreed requirements

- A registration form.
- One registration per student per event.
- Duplicate detection.
- Editing existing registrations.
- Confirmation emails.
- A confirmation screen.

Everyone confirms the specification.

Why this is a good demo

It shows a real problem, identifies several distinct gaps, and produces a useful outcome.

More importantly, the app doesn't have to build the registration website itself. It only needs to demonstrate how it helps the team agree on what to build.

That keeps the MVP manageable while still showing meaningful AI functionality.

9. Technical architecture

For a team of three, the architecture should be simple enough to build, debug, and deploy within the event.

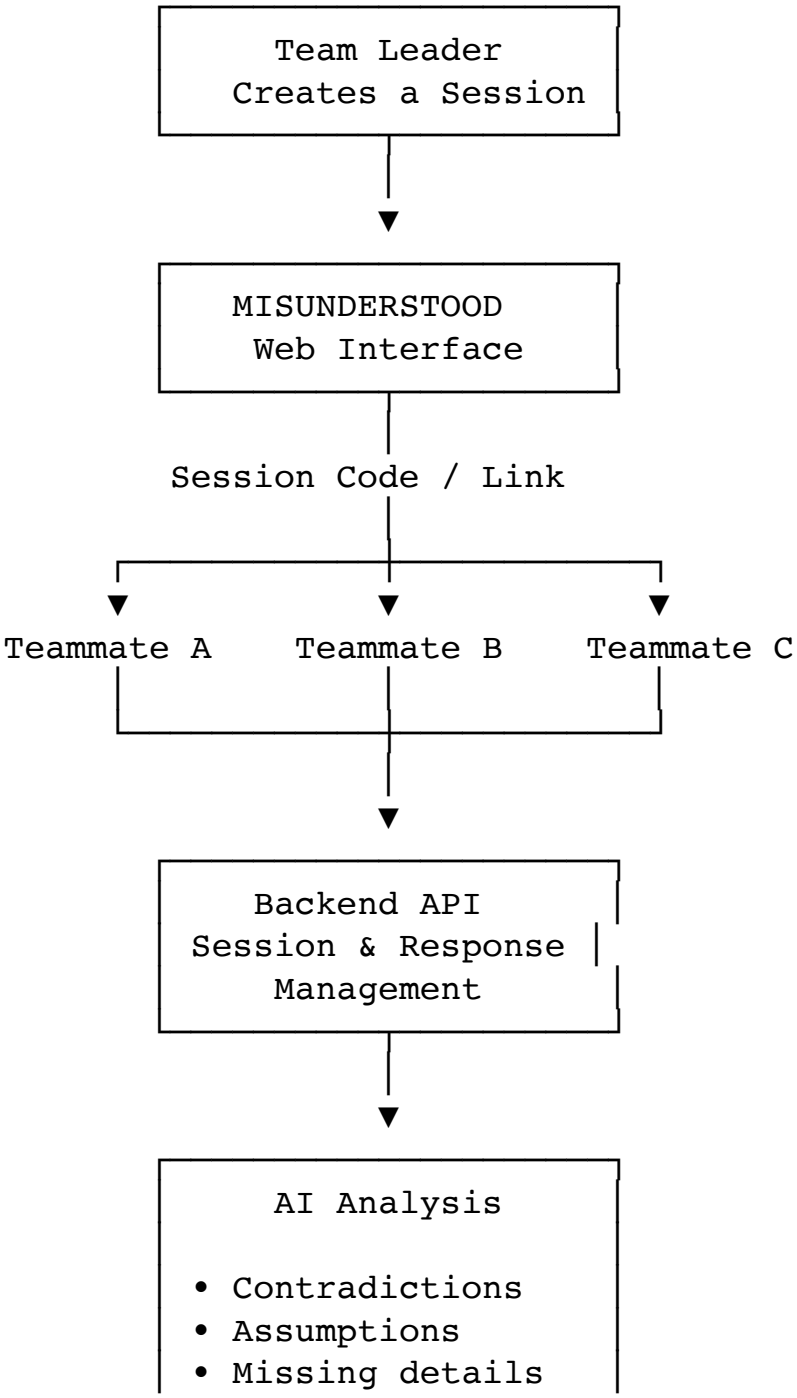
Recommended stack

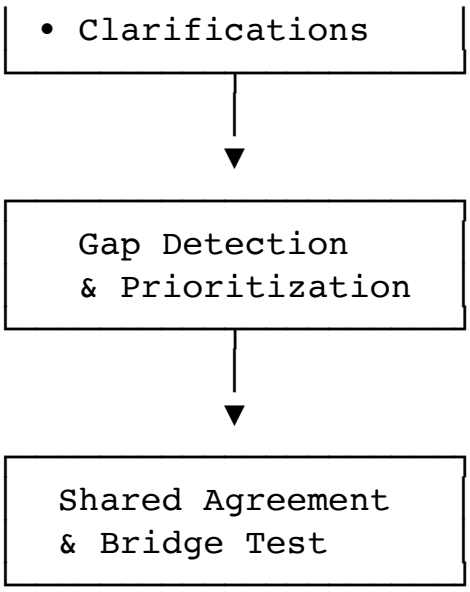
Layer	Technology	Responsibility
Frontend	React + Vite	User interface and session workflow
Styling	Tailwind CSS	Responsive design and visual consistency
Backend	FastAPI or a lightweight serverless backend	Session management and analysis orchestration

AI	An available LLM API	Extract interpretations and identify potential gaps
Database	Supabase/PostgreSQL	Store sessions, responses, issues, and agreements
Deployment	Vercel or equivalent	Host the frontend
Backend hosting	A compatible hosting service	Run the backend if required

You do not need to use all of these technologies. If your team already knows a different stack, use what you can build with confidently.

System architecture





How the AI analysis should work

I recommend a two-stage process.

Stage A: Extract structured interpretations

Send the original task and each participant's response to the model.

Ask it to identify:

- Requirements.
- Assumptions.
- Constraints.
- Expected outcomes.
- Uncertainties.
- Evidence from the participant's actual response.

The model returns structured data.

Stage B: Compare interpretations

Compare the extracted information across participants.

Identify:

- Explicit contradictions.
- Unconfirmed assumptions.
- Missing requirements.
- Complementary contributions.
- Questions that would resolve important ambiguities.

For example, the AI should not conclude that two people disagree merely because they use different words.

"Users should remain signed in" and "users should not have to log in every time they reopen the app" may describe the same intended outcome.

The model should compare meaning rather than matching phrases.

Reliability measures

To make the system useful:

1. **Require evidence:** Each detected issue should link to the statements that triggered it.
2. **Allow corrections:** Participants must be able to reject an incorrect AI interpretation.
3. **Ask rather than assume:** If the system lacks enough information, it should request clarification.
4. **Track resolution:** An issue should remain unresolved until the participants agree on the answer.
5. **Keep the score transparent:** Explain why an issue is classified as important or critical.

The AI should assist the team in reaching agreement, not dictate what the team must believe.

10. Database design

You don't need a complicated database. Four main entities are enough for the first version.

A. Sessions

Stores the task and session information.

Field	Purpose
`id`	Unique session identifier
`title`	Session name
`task_description`	Original instruction
`session_code`	Code used to join
`status`	Collecting responses, analyzing, or resolved
`created_at`	Session creation timestamp

B. Participants

Stores information about each participant in a session.

Field	Purpose
`id`	Participant identifier
`session_id`	Associated session
`display_name`	Name displayed in the interface
`role`	Developer, designer, project lead, etc.
`joined_at`	Join timestamp

C. Responses

Stores each participant's original interpretation.

Field	Purpose
`id`	Response identifier
`session_id`	Associated session
`participant_id`	Person who submitted it
`response_text`	Original interpretation
`submitted_at`	Submission timestamp

D. Gap issues

Stores the issues identified during analysis.

Field	Purpose
`id`	Issue identifier
`session_id`	Associated session
`issue_type`	Contradiction, assumption, missing information, or complementary perspective
`description`	Explanation of the issue
`evidence`	Relevant participant statements
`priority`	Minor, important, or critical
`status`	Open, clarified, or resolved
`resolution`	Agreed outcome, if resolved

You can represent the shared agreement as a collection of resolved issues and confirmed requirements rather than creating another complex subsystem.

Privacy recommendation: Store only the information necessary for the session. Avoid collecting sensitive personal conversations, and give participants a clear way to delete their session data.

11. How three team members should divide the work

With a team of three, avoid having everyone work on the same feature. Assign clear ownership and integrate early.

Member 1: Frontend and UI/UX

Primary responsibility: Make the product intuitive and visually impressive.

Tasks:

- Build the landing page.

- Create and join session screens.
- Build the independent-response interface.
- Design the Misunderstanding Map.
- Build the clarification interface.
- Create the shared agreement screen.
- Build the Bridge Test dashboard.
- Ensure the interface works on different screen sizes.

Deliverable: A complete, navigable user experience.

Member 2: Backend and database

Primary responsibility: Make the application work reliably.

Tasks:

- Create session APIs.
- Implement session codes.
- Manage participant responses.
- Set up the database.
- Store gap issues and resolutions.
- Implement session status transitions.
- Connect the frontend to the backend.
- Deploy and test the application.

Deliverable: A functioning session and data-management system.

Member 3: AI and integration

Primary responsibility: Build the core differentiating feature.

Tasks:

- Design the interpretation-extraction prompt.
- Create structured AI outputs.
- Detect contradictions and unconfirmed assumptions.
- Generate clarification questions.
- Implement evidence-linked issues.
- Build the prioritization logic.
- Integrate the AI analysis into the backend.
- Test the model against deliberately ambiguous examples.

Deliverable: An AI analysis pipeline that produces useful, structured results.

How the team should collaborate

Time	Frontend member	Backend member	AI member
Hours 0–2	Design screens and user flow	Define database and API contracts	Define analysis schema
Hours 2–6	Build core interface	Implement sessions and responses	Build extraction prototype
Hours 6–10	Build results screen	Connect database and frontend	Implement gap detection
Hours 10–14	Build clarification workflow	Integrate analysis API	Improve comparison logic

Hours 14–18	Build agreement and test screens	Integrate and deploy	Test edge cases
Hours 18–21	Polish UI	Fix integration issues	Improve output reliability
Hours 21–24	Rehearse demo	Verify deployment	Prepare test cases and demonstration

These are approximate milestones, not rigid individual schedules.

Important: Agree on the data structures and API contract during the first two hours. This prevents the frontend, backend, and AI components from producing incompatible outputs.

12. The actual 24-hour MVP

Your biggest risk is trying to build too much.

I would define the MVP as one complete journey:

Create a session → Collect independent interpretations → Detect gaps → Clarify differences → Confirm an agreement → Show the improvement.

Everything else is optional.

Phase 1: Foundation — Hours 0–4

Build:

- Landing page.
- Session creation.
- Join-by-code functionality.
- Basic response submission.

At the end of this phase, three teammates should be able to submit their interpretations of one task.

Phase 2: Core AI — Hours 4–10

Build:

- Structured interpretation extraction.
- Contradiction detection.
- Assumption detection.
- Missing-requirement detection.
- Evidence-linked issue output.

At the end of this phase, the application should identify meaningful differences between at least two participants.

Phase 3: Resolution — Hours 10–15

Build:

- Misunderstanding Map.

- Clarification questions.
- Issue status updates.
- Shared agreement generation.
- Participant confirmation.

At the end of this phase, the team should be able to resolve an issue and see the agreed outcome.

Phase 4: Bridge Test — Hours 15–18

Build:

- Before-and-after metrics.
- Unresolved issue counts.
- Confirmed requirement counts.
- A final session summary.

Phase 5: Polish and testing — Hours 18–21

Test:

- Responses that agree despite different wording.
- Responses that contradict one another.
- Missing requirements.
- Ambiguous instructions.
- Incorrect AI interpretations.
- Failed or delayed API requests.

Add sensible loading states and error messages.

Phase 6: Final preparation — Hours 21–24

- Deploy the application.
- Test it using the actual presentation setup.
- Prepare a reliable demonstration session.
- Rehearse the pitch.
- Prepare a backup video or screenshots in case the live AI service fails.

Avoid adding major features during the final hours.

13. What makes MISUNDERSTOOD different?

Let's be precise about our positioning.

Existing products already handle meeting summaries, writing assistance, requirement management, and clarification. We should not claim that MISUNDERSTOOD is the first AI tool to identify misunderstandings.

Our proposed differentiators are how we combine these capabilities into a single workflow.

Common approach	MISUNDERSTOOD's intended approach
Summarize what people said	Compare what each person understood
Improve the wording of a message	Identify unresolved interpretations

Generate a list of requirements	Identify which requirements are not mutually agreed upon
Present an AI-generated answer	Show evidence from each participant's response
Let users ask follow-up questions	Generate questions targeted at specific gaps
End with a summary	End with an explicit, participant-confirmed agreement
Claim to improve communication	Demonstrate the change using a before-and-after test

The strongest differentiator is the **complete process from independent interpretation to verified agreement**.

That's the product we should build and present.

14. Potential judging advantages

Devlympics evaluates theme interpretation, fun and experience, creativity and originality, execution, presentation, and fun twists. Event Design Devlympics

Here's how MISUNDERSTOOD can address each criterion.

Theme interpretation

The project connects people separated by different interpretations, assumptions, and expectations.

Creativity and originality

The product focuses on invisible gaps between people's mental models rather than merely translating or transmitting information.

Execution

The MVP has a clear technical core: structured interpretation extraction, evidence-linked gap detection, clarification, and agreement generation.

Fun and experience

The interface can use visual conflict cards, a live Misunderstanding Map, and an interactive Bridge Test.

Presentation

The demo can begin with a relatable misunderstanding and reveal the consequences before showing how the product resolves it.

Fun twists

The product can incorporate humorous ambiguity examples, teammate-generated interpretations, and an intentionally confusing instruction as a demonstration.

None of these guarantees a high score. The actual result will depend on the quality of the implementation and the judges' evaluation.

15. Suggested UI design

I recommend a clean, modern interface with a distinctive visual identity.

Landing page

Headline:

"Same conversation. Different realities."

Subheadline:

"An AI-powered workspace that reveals hidden assumptions, identifies conflicting interpretations, and helps teams agree on what they actually mean."

Primary button: **Start Bridging**

Secondary button: **Join a Session**

Show a small visual example in which one instruction produces three different interpretations.

Session page

Include:

- Task description.
- Participant list.
- Response submission area.
- Session progress indicator.

Before submission, participants shouldn't see the other responses.

Misunderstanding Map

Organize issues into clear categories:

- Contradictions.
- Unconfirmed assumptions.
- Missing requirements.
- Complementary perspectives.

Each issue should show the relevant statements, the reason it was flagged, and a button to resolve it.

Resolution page

Show the clarification question and allow participants to:

- Select an option.
- Suggest an alternative.
- Confirm the final requirement.

Final agreement page

Display:

- The agreed requirements.
- Unresolved issues.
- Participant confirmations.
- The final shared checklist.
- The Bridge Test results.

This gives the application a coherent beginning, middle, and end.

16. A 90-second presentation script

Here's a pitch you can adapt for your final presentation.

[0–15 seconds: The problem]

"Have you ever told someone something, watched them nod, and later discovered they understood something completely different?

It happens everywhere—in college projects, workplaces, and software development.

The problem isn't always poor communication. Sometimes, it's the false belief that communication has succeeded."

[15–30 seconds: The demonstration]

"Imagine telling three teammates to prepare a project for launch.

One thinks the application must be deployed. Another thinks the interface must be polished. The third thinks testing and documentation are required.

Everyone says, 'Got it.'

But do they actually agree?"

[30–50 seconds: The solution]

"Meet MISUNDERSTOOD.

Each teammate independently explains what they understood. Our AI compares their interpretations, identifies conflicting assumptions, and highlights missing requirements.

Instead of another generic summary, we show the team exactly where their understanding differs."

[50–70 seconds: The live demo]

"Here, our AI has identified a conflict about the launch requirements.

It explains the difference, asks a targeted clarification question, and lets the team agree on what launch readiness actually means.

The application then generates a shared checklist that every participant can confirm."

[70–82 seconds: The impact]

"Our Bridge Test compares the unresolved issues before and after clarification.

We don't just claim that communication improved. We show which gaps were identified and how they were resolved."

[82–90 seconds: The closing]

"MISUNDERSTOOD doesn't just help people communicate.

It helps them discover when they aren't understanding each other.

Because before you say 'Got it,' make sure you actually got it."

17. Final project specification

Here's the version I would commit to for Devlympics.

Project name: MISUNDERSTOOD

Theme: Bridge the Gap

Problem statement: People frequently interpret the same instruction differently, leading to conflicting assumptions, incomplete requirements, and avoidable rework.

Solution: An AI-powered collaboration platform that collects independent interpretations, identifies discrepancies, generates clarification questions, and helps participants establish a shared understanding.

Target users: Student project teams, software developers, designers, project managers, and collaborators working across different levels of knowledge or expertise.

Primary differentiator: The Assumption Gap Detector, supported by evidence-linked issues and participant-confirmed resolutions.

Supporting features:

1. Independent interpretation collection.
2. AI-powered gap detection.
3. Misunderstanding Map.
4. Prioritized unresolved issues.
5. Clarification workflow.
6. Shared agreement generation.
7. Before-and-after Bridge Test.

MVP objective: Demonstrate that the system can identify genuine discrepancies in a shared task, help participants resolve them, and record the agreed outcome.

Team structure: One frontend developer, one backend developer, and one AI/integration developer.

Success criterion for the hackathon: A working end-to-end demo that reliably demonstrates the core workflow—not the number of features or the complexity of the technology stack.

One final technical decision

I recommend starting with **typed responses rather than live audio or meeting transcription**.

Typed responses are easier to implement, debug, evaluate, and demonstrate within 24 hours. Once the core system works reliably, voice input can be added as a future extension.

This keeps the focus on what makes MISUNDERSTOOD valuable: identifying the gap between what people think they understand and what they have actually agreed upon.